CHAPTER 07

Software Maintenance

ACRONYMS

API	Application Programming Interface
CI	Continuous Integration
IEC	The International Electrotechnical Commission
IEEE	The Institute of Electrical and Electronics Engineers
ISO	International Organization for Standardization
KA	Knowledge Area
LOC	Lines of Code
MR	Modification Request
PR	Problem Report
SCM	Software Configuration Management
SEE	Software Engineering Environment
SLA	Service-Level Agreement
SLI	Service-Level Indicators
SLO	Service-Level Objectives
SQA	Software Quality Assurance
V&V	Verification and Validation
XaaS	Anything as a Service

INTRODUCTION

Successful software development efforts result in the delivery of a software product that satisfies user requirements. As those requirements and other factors change, the software product must evolve: Once the software is in operation, defects are uncovered, operating environments change, and new user requirements surface. The maintenance phase of the life cycle begins after a warranty period or after post-implementation support delivery, but maintenance activities occur much earlier.

Software maintenance is an integral part of a software life cycle. However, it has not received the same degree of attention as the other software engineering activities. Historically, software development has had a much higher profile than software maintenance. This is now changing as organizations strive to optimize their software engineering investment by ensuring continuous development, maintenance and operation, progressively eliminating the organizational silos among these areas. The growing acceptance of DevOps practices and tools have drawn further attention to the need to continuously evolve software while ensuring its smooth operation to satisfy users, who are demanding quicker turnaround from software engineers than in the past.

In this *SWEBOK Guide*, software maintenance is defined as the totality of activities required to provide cost-effective support for software in operation. Activities to support software operation and maintenance are performed during the pre-delivery stage and during the post delivery stage. Pre-delivery activities include planning for post-delivery operations, maintainability and determining the logistics support needed for the transition from development to maintenance. Post-delivery activities include software surveillance, modification, training, and operating or interfacing with a help desk.

The Software Maintenance knowledge area (KA) is related to all other aspects of software engineering. Therefore, this KA description is linked to all other software engineering KAs in the *Guide*.

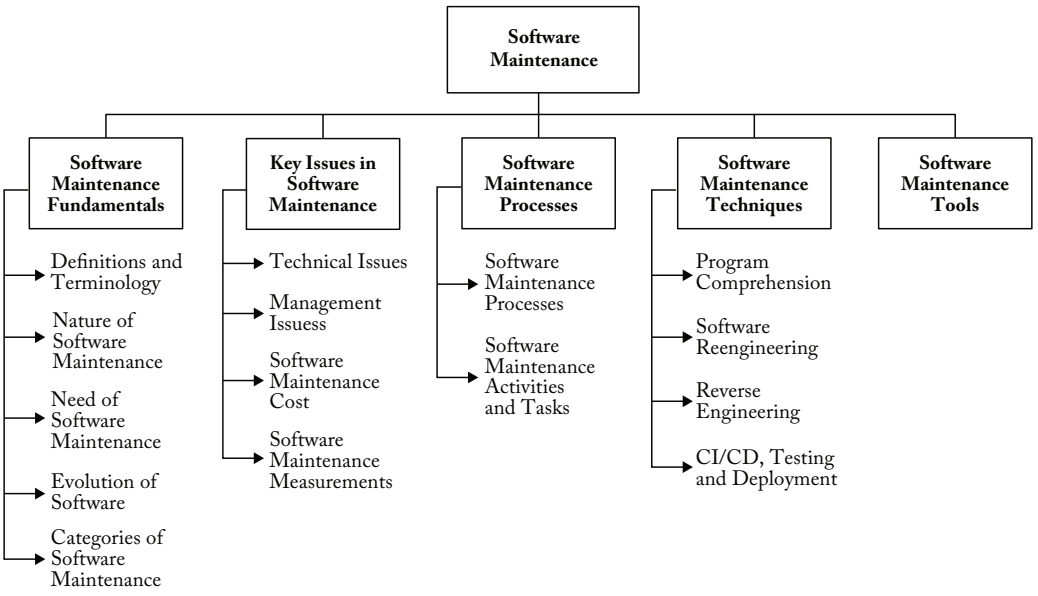

Figure 7.1. Breakdown of Topics for the Software Maintenance KA

BREAKDOWN OF TOPICS FOR SOFTWARE MAINTENANCE

The breakdown of topics for the Software Maintenance KA is shown in Figure 7.1.

1. Software Maintenance Fundamentals

This section introduces the concepts and terminology that form a basis for understanding the role and scope of software maintenance. Among these concepts are the different categories of software maintenance, which are essential to understanding this knowledge area.

1.1. Definitions and Terminology

[1, s3.1][2*, c1s1.2, c2s2,2]

The purpose of software maintenance is defined in the international standard for software maintenance: ISO/IEC/IEEE 14764 [1]. In the context of software engineering, software maintenance is essentially one of many technical processes. The objective of software maintenance is to modify existing software while preserving its integrity. The international standard also emphasizes the importance of performing some maintenance activities before final delivery of the software (pre delivery activities). Software maintenance shares knowledge and tools with software development and software operation and also has its own processes and techniques.

1.2. Nature of Software Maintenance

[2*, c1s1.3]

Software maintenance sustains the software product throughout its life cycle (from development through operations and eventual retirement). The software is monitored for capacity, continuity and availability. Modification requests (MRs) and incident or problem reports (PRs) are logged and tracked, the impact of proposed changes is determined, code and other software artifacts are modified, testing is conducted, and a new version of the software product is released into operation. Also, training and daily ongoing support are provided to users. A *software maintainer* is defined as a role or an organization that performs software

maintenance activities. In this KA, the term sometimes refers to individuals who perform those activities, to contrast their role with the software developer's role.

Maintainers can learn from the developers' and operators' knowledge of the software. Early contact with the developers and early involvement by the maintainers can reduce the overall maintenance costs and efforts. An additional challenge is created when maintainers join the project after the initial developers have left or are no longer available. Maintainers must understand and use software artifacts from development (e.g., code, tests or documentation), support them immediately, and progressively evolve and maintain them over time.

1.3. Need for Software Maintenance

[2*, c1s1.5]

Software maintenance is needed to ensure that the software continues to satisfy user requirements throughout its life span. Maintenance is necessary regardless of the type of software life cycle model used to develop it (e.g., waterfall or Agile). Software products change as a result of both corrective and non-corrective actions. Software maintenance is typically performed to do the following:

- Correct faults and latent defects
- Improve the design or performance of operational software
- Implement enhancements
- Help users understand the software's functionality
- Adapt to changes in interfaced systems or infrastructure
- Prevent security threats
- Remediate technical obsolescence of system or software elements
- Retire the software

1.4. Majority of Maintenance Costs

[2*, c4s4.3, c5s5.2]

It is generally accepted that the relative cost of error fixing increases in later phases of the software life cycle. Maintenance also uses a significant portion of the total financial resources attributed throughout the life of a software. A common perception of software maintenance is that it merely fixes faults. However, studies and surveys over the years have indicated that most software maintenance — over 80% — is used for enhancing and adapting the software [3]. Grouping enhancements and corrections together in management reports contributes to a misconception that corrections cost more than they really do. Understanding the categories of software maintenance helps us understand the structure of software maintenance costs — that is, where most of that spending goes [7]. Also, understanding the factors that affect the maintainability of software can help organizations contain costs. Environmental factors that affect software maintenance costs include the following:

- Operating environment (hardware and software).
- Organizational environment (policies, competition, process, product and personnel).

1.5. Evolution of Software

[2*, c3s3.5]

Software maintenance as an activity that supports the evolution of software was first addressed in the late 1960s. Research, by Lehman and others [8], over a period of twenty years led to the formulation of eight laws of software evolution:

- Continuing Change — Software must be continually adapted, or it becomes progressively less satisfactory.
- Increasing Complexity — As software evolves, its complexity increases unless work is done to maintain or reduce that complexity.
- Self-Regulation — The program evolution process is self regulating with close to normal distribution of measures of product and process attributes.

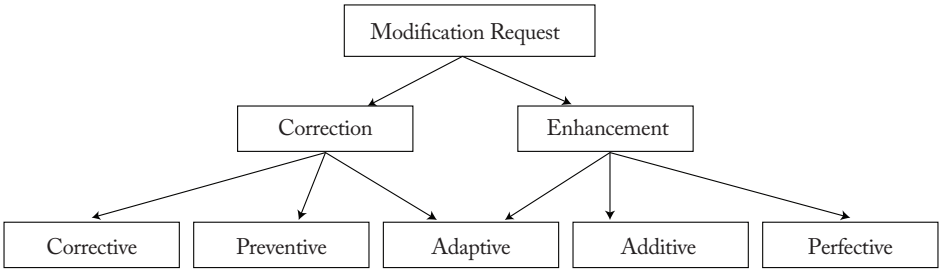

Figure 7.2. Software Maintenance Categories

- Invariant Work Rate — The average effective global activity rate in an evolving software package is invariant over the product's lifetime.
- Conservation of Familiarity — As software evolves, all associated with it (e.g., developers, sales personnel and users) must maintain mastery of its content and behavior to achieve satisfactory evolution. Excessive growth diminishes that mastery. Hence, average incremental growth remains invariant as the system evolves.
- Continuing Growth — Functional content of a program must be continually increased to maintain user satisfaction over its lifetime.
- Declining Quality — The quality of software will appear to be declining unless it is rigorously maintained and adapted to changes in the operational environment.
- Feedback System — Software evolution processes constitute multilevel, multiloop, multi-agent feedback systems and must be treated as such to achieve significant improvement over any reasonable base.

Key findings of Lehman's research include a proposal that maintenance is evolutionary development and that maintenance decisions are aided by an understanding of what happens to software over time. Another way to think of maintenance is as continued development that accommodates extra inputs (or constraints) — in other words, large software programs are never complete and continue to evolve. As they evolve, they grow more complex unless action is taken to reduce that complexity.

1.6. Categories of Software Maintenance

[1, s3.1.8][2*, c1s1.8, c3s3.3]

Five categories (types) of software maintenance have been standardized to classify a maintenance request: corrective, preventive, adaptive, additive and perfective. ISO/IEC/IEEE 14764 [1], regroups these maintenance categories as either corrections or enhancements, as shown in Figure 7.2.

ISO/IEC/IEEE 14764 [1] also defines a sixth category — emergency maintenance:

- Corrective maintenance: Reactive modification (or repairs) of a software product performed after delivery to correct discovered problems.
- Preventive maintenance: Modification of a software product after delivery to correct latent faults in the software product before they occur in the live system.
- Adaptive maintenance: Modification of a software product performed after delivery to keep a software product usable in an evolving environment (e.g., an upgrade to the operating system results in changes to the applications).
- Additive maintenance: Modification of a software product performed after delivery to add functionality or features to enhance the usage of the product.

Additive maintenance differs from perfective maintenance in that a) it provides additional new functions or features to improve software usability, performance, maintainability or other software quality attributes, and b) it adds functionality or features with relatively large additions or changes for improving software attributes after delivery.

- Perfective maintenance: Modification of a software product after delivery to provide enhancements for users, improvement of program documentation, and recoding to improve software performance, maintainability, or other software attributes.
- Emergency maintenance: Unscheduled modification performed to temporarily keep a system operational, pending corrective maintenance.

2. Key Issues in Software Maintenance

A number of key issues must be dealt with to ensure the effective maintenance of software. Software maintenance provides unique technical and management challenges for software engineers (e.g., the challenge of finding a fault in large complex software developed by someone else.)

Similarly, in an Agile setting, maintainers and developers are constantly striving to make sure that clients see the value at the end of each iteration so maintenance activities have to compete with the development of new features for client approval; Planning for a future release, which often includes coding the next release while sending out emergency patches for the current release, also creates a challenge in balancing maintenance and development work. The following section presents technical and management issues related to software maintenance. They are grouped under the following topics:

- Technical issues.
- Management issues.
- Software maintenance costs.
- Software maintenance measurement.

2.1. Technical Issues

2.1.1 Limited Understanding

[2*, c6s6.9]

Limited understanding describes a software engineer's initial comprehension of software someone else developed. This is reflected in how quickly a software engineer can understand where to change or correct the software. Research suggests a significant portion of total maintenance effort is devoted to understanding the software to be modified. Consequently, the topic of software comprehension is of great interest to software engineers. A number of comprehension factors have been identified: 1) domain knowledge; 2) programming practices (e.g., implementation issues); 3) documentation; and 4) organisation and presentation issues. Comprehension is more difficult in text-oriented representation (e.g., in source code), where it is often difficult to trace the evolution of software through its releases or versions if changes are not documented and the developers are not available to explain them. Thus, software engineers may initially have a limited understanding of the software, and much work must be done to remedy this. Various techniques can help engineers understand existing software, such as visualization and reverse engineering using tool-based graphical representations of the code.

2.1.2 Testing

[1, s6.2][2*, c9, c13s13.4.4]

Test planning and activities occur during MRs and PRs processing. The cost of repeating full testing on a major piece of software is significant, in both time and effort. To ensure a software modification is validated, the maintainer should replicate or verify changes by planning and executing the appropriate tests — for example, regression testing is important in maintenance. Regression testing is the selective retesting of software or a component to verify that the modifications have not caused unintended effects. Another challenge is

finding the time to conduct as much testing as possible. Coordinating tests can be challenging for maintenance team members who are simultaneously working on different problems. Bringing software offline to test it can be difficult if the software performs critical functions. The Software Testing KA provides additional information and references on software testing and its subtopic on regression testing.

2.1.3 Impact Analysis

[1, s5.1.6][2*, c13s13.3]

Impact analysis assesses the detailed effects of proposed changes on existing software. Software engineers should strive to conduct the analysis as cost-effectively as possible. Maintainers need detailed knowledge of the software's structure and content. They use that knowledge to perform the impact analysis, which identifies all systems and software products that would be affected by a software change request and develops an estimate of the resources needed to accomplish the change. The analysis also determines the risks involved in making the change. The change request (originating from an MR or a PR) must first be analyzed and translated into software terms. Impact analysis is performed after a change request enters the software configuration management (SCM) process. ISO/IEC/IEEE 14764 [1] states that the impact analysis tasks do the following:

- Develop an identification scheme for MRs/PRs.
- Develop a scheme for categorizing and prioritizing MRs/PRs.
- Determine the procedures for an operator to submit an MR/PR.
- Identify the information needs and issues that must be tracked and reported to the users and identify the measures that provide feedback on those information needs and issues.
- Determine how temporary work-arounds will be provided to the operators.
- Track the work-around(s) through to removal.
- Determine what follow-up feedback will be provided to the users.

Software maintainers often use the severity of a PR as a guide when deciding how and when to fix the problem. The maintainer conducts an impact analysis that identifies the affected components, develops several potential solutions, and, finally, recommends a course of action.

Impact analyses of proposed maintenance changes often consider various factors such as the maintenance category, the size of the modification, the cost involved, the testing needed to make the modification, and any impacts on performance, safety and security. Designing software with maintainability in mind greatly facilitates impact analysis. More information can be found in the Software Configuration Management KA.

2.1.4 Maintainability

[1, s8.8][2*, c12s12.5.5]

ISO/IEC/IEEE 14764 [1] defines maintainability as the capability of the software product to be modified. Modifications can include corrections, improvements or adaptation of the software to changes in environment, as well as changes in requirements and functional specifications.

As an important software quality characteristic, maintainability should be specified, reviewed and controlled during software development activities in order to reduce maintenance costs. When these activities are carried out successfully, the software's maintainability will benefit. Maintainability is often difficult to achieve because it is often not a primary focus during software development. The developers are typically more focused on other activities and might not pay enough attention to maintainability requirements. This can result in bad architecturing, missing software documentation or test environments, which is a leading cause of difficulties in program comprehension and subsequent impact

analysis during maintenance. The presence of systematic and mature software development processes, techniques and tools helps enhance the maintainability of software. The Software Quality KA provides additional information and references on software maintainability.

Compromised software maintainability typically increases the burden on software engineers who maintain the software in the future; in other words, it creates technical debt. Technical debt often accumulates when the need to quickly address corrective, emergency, and additive maintenance tasks, constrained by limited time and understanding of the software, leads to compromises. These immediate but potentially under-considered solutions, often not peer-reviewed, contribute to the accumulation of technical debt. This practice generally creates a technical debt that will take additional time and effort to address during maintenance. Specifically, software engineers must investigate three areas in depth when addressing technical debt:

1. Code quality versus relevance: Not all technical debt is urgent.
2. Alignment with organizational objectives: The software architecture should reflect the organization's goals.
3. Process loss: Ensure complementary skills of software engineers involved.

2.2. Management Issues

2.2.1. Alignment with Organizational Objectives [1, s9.1.8][2*, c2s2.3.1.2, c3s3.4]

This section describes how to optimize software maintenance activities and economics to be aligned with organizational objectives and the priorities of the business, customers and users.

In many organizations, initial software development is project-based, with a defined time scale and budget. The main goal is to deliver a product that meets user needs on time and within budget. In contrast, software maintenance aims to extend the life of software and keep it operational for as long as needed. In addition, it may be driven by the need to meet user demand for software updates and enhancements.

In both cases, the economics of software maintenance is not as visible as those of software development. At the organizational level, it may be seen as an activity that consumes significant resources with no clear, quantifiable benefit for the organization. As a consequence, adding new features is often given higher priority than other maintenance activities (such as refactoring, security or performance improvement) to meet the goals and objectives of software customers, as well as with constraints such as time and budget. However, such organizational objectives and constraints must be balanced with software maintainability and engineering standards to avoid code decay and technical debt.

Applying product management approaches to the management of software development and maintenance can help organizations:

- Understand the total cost of operational software over its full life cycle.
- Compare the costs and benefits of developing new software versus enhancing existing software.
- Resolve staffing and skills issues, as the same team can be responsible for maintenance and development.
- Focus more on maintainability requirements from the start, as the same team has responsibility for both development and maintenance.

2.2.2. Staffing [1*, s6.4.13.3c] [2*, c2s2.3.1.5, c10s10.4]

Although maintenance work is sometimes perceived as less engaging, this view overlooks the critical importance of software maintainers. Given that maintenance constitutes a significant portion of software lifecycle activities, recognizing and valuing the contribution of maintainers is essential to boosting morale, performance, and reducing staff turnover.

Organizations need to design development and maintenance teams and roles carefully and provide professional development opportunities for their staff.

2.2.3. Process [1*, s6][2*, c5]

The software life cycle process is a set of activities, methods, practices and transformations that people use to develop and maintain software and its associated products. At the process level, software maintenance activities share much in common with software development (e.g., SCM is a crucial activity in both). Maintenance also requires several activities not found in software development. (Refer to section 3.2.)

2.2.4. Supplier Management [1*, s6.1.2, s8.3, s8.8.2]

Supplier management ensures that the organization's suppliers and their performance are managed appropriately to support the seamless provision of quality products and services when maintenance is contracted to suppliers. The nature of the organization's relationship with suppliers and its approach to supplier management should be determined by the nature of these products and services. Contractors can be hired to conduct maintenance tasks and outsourcing or offshoring software maintenance is a major industry. Outsourcing maintenance means substituting internal capability with an external supplier's capability. Approaches to contracting maintenance include the following:

- Single source or partnership: A single supplier provides all services, or an external service integrator manages the organization's relationship with all suppliers.
- Multi-sourcing: Products and services are provided by more than one independent supplier. These are combined into a single (software-enabled) service. Multi-sourcing in software services is increasingly common, enabled by the growth of "anything as a service" (XaaS), application programming interfaces (APIs), and data sources.

Many organizations outsource entire portfolios of software. Typically, these portfolios include software that is not mission-critical, as organizations do not want to lose control of the software used in their core business. One major challenge for outsourcers is determining the scope of the maintenance services required, the terms of a service-level agreement (SLA), and the contractual details. Outsourcers need to invest in good communication infrastructure and an efficient help desk staffed with people who can communicate effectively with customers and users [3]. Outsourcing requires a significant initial investment and the setup and review of software maintenance processes that require automation.

2.2.5. Organizational Aspects of Maintenance [1, s9.1.8][2*, c10]

Organizational aspects of maintenance include determining which teams will be responsible for software maintenance. When using Agile life cycle models, the developer also conducts maintenance tasks, acting as both developer and maintainer. Other organizations prefer that the team that develops the software does not necessarily maintain it once it is operational. In deciding where the software maintenance function will be located, software engineering organizations must consider each alternative's advantages and disadvantages. There are a number of disadvantages to having the developer also maintain the software after it has been put into production, such as the risk that new development will be disrupted when the developers need to attend to failures and the potential loss of knowledge when developers leave the organization, since fewer individuals are familiar with the software; this could also lead to lower-quality documentation, as fewer individuals are involved. However, having a separate maintenance function also has its challenges, as many software engineers do not like limiting their work to maintenance and may be more likely to

leave for more interesting work. In addition, a handoff process must be put in place between developers and maintainers, which sometimes leads to friction between teams [3].

The introduction of product management processes has encouraged a single-team approach, particularly for developing and maintaining software that needs to respond rapidly to changes in customer and user needs. Because there are many pros and cons to each option, the decision should be made on a case-by-case basis. What is important is that the organization delegates the maintenance tasks to an experienced group or person and keeps quality documentation on maintenance tasks and all changes made to the software, regardless of the organization's structure.

2.3. Software Maintenance Costs

Software engineers must understand the different categories of software maintenance described in 1.6. Presenting costs trends by categories of maintenance can show customers where maintenance effort is spent for each system supported [7]. The data about maintenance effort by category can be also used to accurately estimate the cost of software maintenance. Cost estimation is an important aspect of planning software maintenance.

2.3.1. Technical Debt Cost Estimation

[1, s6.1.7, s8.8.3.6][2*, c12.12.5]

Technical debt generally makes code more expensive to maintain than it has to be. Technical debt represents the effort required to fix problems that remain in the code when an application is initially released by the development team. Several techniques and indicators can help engineers measure technical debt, including, size, complexity and the number of engineering flaws and violations of good architectural design and coding practices in the source code. ISO/IEC/IEEE 14764 provides suggestions for improving maintainability, including: ensuring legibility, pursuing structured code, reducing code complexity, provide accurate code comments, using identation and white space, eliminating language weaknesses and compiler dependent constructs, facilitate error-tracing, ensure traceability of code to design, conduct inspections and code reviews. A software product needs to evolve, by adding new features and capabilities, and its codebase must remain maintainable, easily understood, and easy to further evolve. A common barrier to addressing technical debt — or, indeed, of implementing any potential enhancement — is the uncertain reward for doing so. That's why it's so important for organizations to determine the following:

- The quality of their current software.
- The extent of their technical debt.
- The potential savings from investing in quality enhancement.
- The impact of current quality issues on their business.

Furthermore, technical debt is only one factor of several contributing to excess unplanned work; team or process issues may also need to be understood and addressed. Modern tooling can help detect such issues, which means technical debt should not be handled in isolation but through an examination of its root causes.

2.3.2. Maintenance Cost Estimation

[1, s6.2.2, s9.1.4, s9.1.9-10]
[2*, c12s12.5.6]

An estimate of software maintenance costs should be prepared early in the software planning process [1, c6s1.4]. The costs should be a function of the scope of maintenance activities. ISO/IEC/IEEE 14764 [1, c7s2.4] identifies various factors that should be included, such as the following:

- Travel to user locations.
- Training for maintainers as well as users.
- Cost and annual maintenance for the software engineering environment (SEE) and software testing.
- Personnel costs (e.g., salaries, benefits).
- Other resource costs, such as consumables.
- Software license maintenance costs.

- Product changes, program management.
- Field service engineers.
- Renting facilities for maintenance.

Moreover, as the maintenance and development efforts progress, the estimates should be amended. Historical measurement data should be used as inputs to estimate maintenance costs. Additionally, cost estimates are also required during impact analysis of individual MR or PR. The cost estimating method (e.g., parametric model, comparison to analog systems, use of empirical and historical data) should be described. Estimates of individual MRs or PRs typically include the estimated effort associated with executing a change, resource estimates and an estimated timeline for implementing the change.

2.4. Software Maintenance Measurement

[1, s6.1.7][2*, c12]

Measurable software maintenance artifacts include maintenance processes, resources and products [2*, c12s12.3.1]. Measures include size, complexity, quality, understandability, maintainability and effort. One useful measure is the amount of effort (in terms of resources) expended for corrective, preventive, adaptive, additive and perfective maintenance.

Complexity and technical debt measures of software can also be obtained using available tools. These measures constitute a good starting point for the measurement of software quality. Maintainers should determine which measures are appropriate for a specific organization based on that organization's needs. Software measurement programs are discussed in the Software Engineering Management KA.

The software quality model described in the Software Quality KA describes software product and process measures specific to software maintenance. Measurable characteristics of maintainability include the following:

- Modularity measures the degree to which a system or software is composed of components that are independent, such that a change to one component has minimal impact on other components.
- Reusability measures how well a component can be reused.
- Analyzability measures the effort or resources the maintainer must expend either to diagnose deficiencies or causes of failure or to identify components to be modified.
- Modifiability measures the maintainer's effort associated with implementing a specified modification without introducing defects or degrading existing product quality.
- Testability measures the effort maintainers and users expend to test the modified software.
- Supportability measures the ease with which support can be provided for the software, encompassing the availability and accessibility of documentation, tools, and assistance for addressing issues, facilitating effective maintenance and troubleshooting.

Other measures that software maintainers use include the following:

- Reliability: The degree to which a system or software performs specific functions under specified conditions for a specified period, including the following characteristics:
  - Maturity: How well a system or software can meet the need for reliability.
  - Availability: Whether a system or software is operational and accessible.
  - Fault tolerance: How well a system or software operates despite hardware or software faults.
  - Recoverability: How well a system or software can recover data during an interruption or failure.
- Size of the software (e.g., functional size, LOC).
- Number of maintenance requests, by time period.

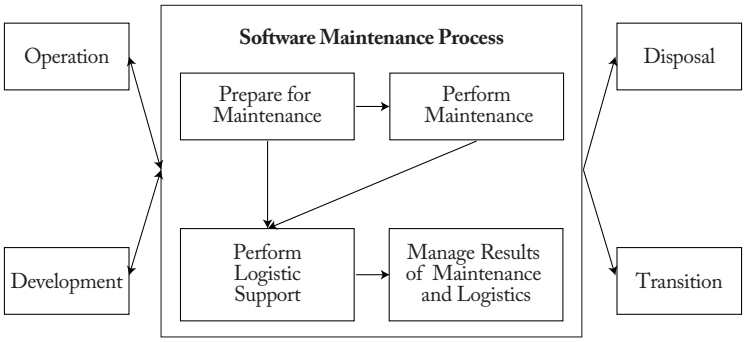

Figure 7.3. Software Maintenance Processes (ISO/IEC/IEEE 14764) [1]

- Effort per maintenance request.
- Software characteristics (e.g., platform, hardware, programming language, frameworks).

Maintenance measures may be collected, analyzed and trended by category to facilitate improvement and to provide insight into where maintenance costs are expended. The degree of software maintenance effort expended for different applications, listed by category, is valuable business information for users and their organizations. It can also enable the organization to make an internal comparison of software maintenance profiles [7].

3. Software Maintenance Processes

In addition to standard software engineering processes and activities described in ISO/IEC/IEEE 14764 [1], a number of activities are unique to maintainers (refer to section 3.2).

3.1. Software Maintenance Processes

[1, s5.2][2*, c5]

Maintenance processes provide needed activities and detailed inputs and outputs to those activities, as described in ISO/IEC/IEEE 14764 [1]. Maintenance is one of the technical life cycle processes presented in ISO/IEC/IEEE 12207 [10]. Figure 7.3 shows how maintenance processes connect to other software engineering processes, which interact to support operational software. The software maintenance processes includes the following:

- Prepare for maintenance.
- Perform maintenance.
- Perform logistics support.
- Manage results of maintenance and logistics.

Recently, Agile methodologies, which promote lightweight processes, have also been adapted to maintenance. This requirement has emerged from the ever-increasing demand for fast turnaround of maintenance services. Improvement to the software maintenance processes is supported by software maintenance maturity models [3].

3.2. Software Maintenance Activities and Tasks

[1, s6.1][2*, c6, c7]

The maintenance process contains the activities and tasks necessary to operate and modify an existing software system while preserving its integrity. These activities and tasks are the responsibility of the operator and the maintainer. As already noted, many maintenance activities are similar to those of software development. Maintainers perform analysis, design, coding, testing and documentation. They must track requirements in their activities — just as in development — and update documentation as baselines change.

ISO/IEC/IEEE 14764 recommends that when a maintainer uses a development process, the process must be tailored to meet specific needs.

However, there are a number of processes, activities and practices that are specialized to software maintenance:

- Program understanding: This comprises the activities needed to obtain a general knowledge of what a software product does and how the parts work together.
- Transition: This is a controlled and coordinated sequence of activities during which software is transferred progressively from the developer to the operations and maintenance team.
- MR acceptance/rejection: Modifications requesting work greater than the agreed size, level of effort, or level of complexity may be rejected by maintainers and rerouted to a developer.
- Maintenance help desk: The help desk is an end-user and maintenance-coordinated support function that triggers the assessment, prioritization and costing of MRs and incidents.
- Impact analysis: The impact analysis identifies areas impacted by a potential change.
- Maintenance service-level indicators (SLIs), service-level objectives (SLOs), SLAs, and maintenance software and hardware licenses and contracts: These are contractual agreements that describe the services and quality objectives of third parties.

3.2.1. Supporting and Monitoring Activities

[s6.4.13.3d5, s6.1.8][2*, c3s3.4]

Maintainers may also perform ongoing support activities, such as documentation, SCM, verification and validation (V&V), problem resolution, software quality assurance (SQA), reviews, vulnerability assessments, and audits. Another important management of maintenance results activity is that of monitoring customer satisfaction.

3.2.2. Planning Activities

[1, s6.1.3, s8.7.2][2*, c10]

An important activity for software maintenance is planning, and this process must address the issues associated with a number of planning perspectives, including the following:

- Business planning (organizational level)
- Maintenance planning (transition level).
- Release/version planning (software level).
- MR planning (at individual request level).

At the individual request level, planning is carried out during the impact analysis. (See section 2.1.3, Impact Analysis.) The release/version planning activity requires that the maintainer do the following:

- Collect the dates of availability of individual requests.
- Agree with users on the content of subsequent releases/versions.
- Identify potential conflicts and develop alternatives.
- Assess the risk of a given release and develop a back-out plan in case problems arise
- Inform all stakeholders.

Whereas software development projects have a typical duration of months to a few years, the maintenance phase usually lasts until the software is retired by the disposal process. Estimating resources is a key element of maintenance planning. Software maintenance planning should begin with the decision to develop a new software product and should consider quality objectives. A concept document should be developed, followed by a maintenance plan, and these should address the following:

- Scope of software maintenance.
- Adaptation of the software maintenance processes and tools.
- Identification of the software maintenance organization.
- Estimate of software maintenance costs.

A software maintenance plan should be prepared during software development and should specify how users will request modifications and report problems or issues. Software maintenance planning is addressed in ISO/IEC/IEEE 14764 [1]. Finally, at the highest level of management, the maintenance organization must conduct software maintenance business planning activities (e.g., communications, budgetary, financial and human resources activities). [2*, c10]

3.2.3. Configuration Management

[1, s6.1.3c, s6.4.13.3d4][2*, c11s11.3]

ISO/IEC/IEEE 14764 [1] describes SCM as an enabling system or service to support the maintenance process. SCM procedures should provide for the verification, validation and audit of each step required to identify, authorize, implement and release the software product and its IT assets undergoing change.

It is not sufficient to track MRs or PRs only. Any change made to the software product and its underlying infrastructure must be controlled. This control is established by implementing and enforcing an approved SCM process. The SCM KA discusses SCM in more detail as well as the process by which change requests are submitted, evaluated and approved. SCM for software maintenance differs from SCM for software development in the number of small changes that must be controlled in the operational environment. The SCM process is implemented by developing and following an SCM plan and operating procedures. Maintainers participate in configuration control boards to determine the content of the next release or version in production.

3.2.4. Software Quality

[1, s6.1.6, s8.7.2][2*, c13s13.4]

It is not sufficient to simply hope that software maintenance will produce higher quality software. Maintainers should have an effective quality program. They must implement processes to support the continuous improvement of software maintenance processes. The activities and techniques for SQA, V&V, reviews, and audits must be selected in concert with all the other processes to achieve the desired level of quality. It is also recommended that both software operations and maintenance adapt and use the output of the software development process, its techniques and deliverables (e.g., test tools and documentation), and test results. More details about software quality can be found in the Software Quality KA.

4. Software Maintenance Techniques

This topic introduces generally accepted techniques used in software maintenance.

4.1. Program Comprehension

[2*, c6, c14s14.5]

Maintainers spend considerable time reading and understanding programs in order to implement changes. Code browsers are key tools for program comprehension and are used to organize and present source code. Clear and concise documentation also aids program comprehension.

4.2. Software Reengineering

[2*, c7]

Software reengineering refers to the examination and alteration of software to reconstitute it in a new form. It includes the subsequent implementation of the new form. It is often undertaken not to improve maintainability but to replace aging legacy software.

Refactoring is a reengineering technique that aims to reorganize a program without changing its behavior. Refactoring seeks to improve the internal structure and the maintainability of software. Refactoring techniques can be used during maintenance activities to reduce the technical debt of the codebase before and after code changes.

In the context of Agile software development, the incremental nature of continuous

integration (CI) often requires the code to be continuously refactored to augment its quality and reliability. Hence, continuous refactoring supports the volatile software life cycle by providing better ways to reduce and manage the growing complexity of software systems while improving developer productivity.

4.3. Reverse Engineering [2*, c7, c14s14.5]

Reverse engineering is the process of analyzing software to identify its components and their interrelationships, as well as creating representations of the software in another form or at higher levels of abstraction. Reverse engineering is passive; it does not change the software or result in new software. Reverse engineering efforts typically produce graphical representations of different software artifacts, such as call graphs and control flow graphs from source code. Types of reverse engineering include the following:

- Re-documentation.
- Design recovery.
- Data reverse engineering — recovering logical schemata from physical databases.

Tools are key for reverse engineering and related tasks such as re-documentation and design recovery. Software visualization is a common reverse engineering technique that helps maintainers explore, analyze and understand the structure of software systems as well as their evolution. Software visualization comprises visually encoding and analyzing software systems, including software maintenance practices, evolution, structure and software runtime behavior using information visualization, computer graphics and human-computer interaction. Generally, software visualization tools are accompanied by various quality assurance features, such as quality metrics calculation, technical debt estimation, and bad design and coding practices (code smells) detection.

4.4. Continuous Integration, Delivery, Testing and Deployment [1, s6.4.13.3 Note 1]

Automating development, operation and maintenance-related tasks saves engineering resources. When implemented appropriately, such automated tasks are generally faster, easier and more reliable than they would be if performed manually. ISO/IEC/IEEE 14764 states that automation includes distribution and installation of software.[1, s6.4.13.3 Note 1]. DevOps supports such automation while building, packaging and deploying reliable and secure systems. DevOps combines development, operations, and maintenance resources and procedures to perform CI, delivery, testing and deployment. [9]

CI is a software engineering practice that continually merges artifacts, including source code updates from all members of a team, into a shared mainline for evolving and testing the developed system. With CI, the members of a team can integrate their changes frequently, and each integration can be verified by an automated build (including testing) to detect integration errors as quickly as possible. The fundamental goal of CI is to automatically catch problematic changes as early as possible. CI helps guarantee the working state of a software system at various points from build to release, thereby improving confidence and quality in software products and improving productivity in teams. Specifically, CI automates the build and release processes with continuous build, continuous delivery, continuous testing and continuous deployment. [6, c23, c24].

Continuous delivery is a software engineering practice that enables frequent releases of new systems (including software) to staging or various test environments through the use of automated tools. Continuous delivery continuously assembles the latest code and configuration to create release candidates.

Continuous testing is a software testing practice that involves testing the software at every stage of the software development life cycle. The goal of continuous testing is to evaluate the quality of software at every step of the

continuous delivery process by testing early and often. Continuous testing involves various stakeholders such as developers, maintainers, DevOps, SQA, and operational systems teams.

Continuous deployment is an automated process of deploying changes to production by verifying intended features and validations to reduce risk. As Martin Fowler, in the book *Continuous Delivery*, pointed out, "The biggest risk to any software effort is that you end up building something that isn't useful. The earlier and more frequently you get working software in front of real users, the quicker you get feedback to find out how valuable it really is."

4.5. Visualizing Maintenance

Maintaining a clear understanding of software systems' evolving structures and dependencies presents a challenge. Visualization is a valuable supporter in software maintenance management, offering a visual representation of the software's components and helping it make informed decisions. With the increasing size and complexity of software systems, visual representations can support software maintenance by enabling dependency analysis, tracing a software evolution history, visualizing software runtime dynamics, and providing complementary documentation. Visualization represents an active research area that synergizes computational capabilities with human pattern detection abilities. It produces visual representations designed to enhance the maintenance team's cognitive performance when faced with complex data analysis.

5. Software Maintenance Tools

[1, c6s4][2*, c14]

This topic encompasses tools that are particularly important in software maintenance where existing software is being modified. Maintenance tools are closely interconnected with development and operations tools. Together, they are part of the SEE. The following are examples of maintenance tools:

- Configuration management, code versioning and code review tools,
- Software testing tools,
- Software quality assessment tools (to assess technical debt and code quality).
- Program slicers, which select only the parts of a program affected by a change.
- Static analyzers, which allow general viewing and summaries of program content.
- Dynamic analyzers, which allow the maintainer to trace the execution path of a program.
- Data flow analyzers, which allow the maintainer to track all possible data flows of a program.
- Cross-referencers, which generate indexes of program components.
- Dependency analyzers, which help maintainers analyze and understand the interrelationships among components of a program.
- Remote Access tools, enabling maintainers to diagnose and modify user systems remotely, crucial for real-time issue resolution and seamless modifications across environments.

Reverse engineering tools support the process by working backward from an existing product to create artifacts such as specification and design descriptions, which can then be transformed to generate a new product from an old one. Maintainers also use software tests, SCM, software documentation and software measurement tools.

MATRIX OF TOPICS VS. REFERENCE MATERIAL

	Grubb and Takang 2003 [2*]
1. Software Maintenance Fundamentals	
1.1. Definitions and Terminology	c1s1.2, c2s2.2
1.2. Nature of Software Maintenance	c1s1.3
1.3. Need for Software Maintenance	c1s1.5
1.4. Majority of Maintenance Costs	c4s4.3, c5s5.2
1.5. Evolution of Software	c3s3.5
1.6. Categories of Software Maintenance	c1s1.8, c3s3.3
2. Key Issues in Software Maintenance	
2.1. Technical Issues	
2.1.1. Limited Understanding	c6s6.9
2.1.2. Testing	c9, c13s13.4.4
2.1.3. Impact Analysis	c13s13.3
2.1.4. Maintainability	c12s12.5.5
2.2. Management Issues	
2.2.1. Alignment with Organizational Objectives	c2s2.3.1.2, c3s3.4
2.2.2. Staffing	c2s2.3.1.5, c10s10.4
2.2.3. Process	c5
2.2.4. Supplier Management	
2.2.5. Organizational Aspects of Maintenance	c10
2.3. Maintenance Costs	
2.3.1. Technical Debt Cost Estimation	c12s12.5
2.3.2. Maintenance Costs Estimation	c12s12.5.6
2.4. Software Maintenance Measurement	c12
3. Software Maintenance Process	
3.1. Software Maintenance Processes	c5
3.2. Software Maintenance Activities and Tasks	c6, c7
3.2.1. Supporting and Monitoring Activities	c3s3.4
3.2.2. Planning Activities	c10
3.2.3. Software Configuration Management	c11s11.3
3.2.4. Software Quality	c13s13.4
4. Software Maintenance Techniques	
4.1. Program Comprehension	c6,c14s14.5
4.2. Software Reengineering	c7

4.3. Reverse Engineering	c7, c14s14.5
4.4. Continuous Integration, Delivery, Testing and Deployment	
4.5. Visualizing Maintenance	
5. Software Maintenance Tools	c14

FURTHER READINGS

A. April and A. Abran, *Software Maintenance Management: Evaluation and Continuous Improvement* [3].

This book explores the domain of continuous software maintenance processes. It provides road maps for improving software maintenance processes in organizations. It describes software maintenance practices organized by maturity levels, which allow for benchmarking and continuous improvement. Goals for each key practice area are provided, and the process model presented is fully aligned with the architecture and framework of international standards ISO/IEC/IEEE 12207, ISO/IEC/IEEE 14764 and ISO/IEC/IEEE 15504, as well as models such as ITIL and CoBIT.

IEEE Std 2675-2021, IEEE Standard for DevOps: *Building Reliable and Secure Systems Including Application Build, Package and Deployment* [5].

Technical principles and processes to build, package, and deploy systems and applications in a reliable and secure way are specified. Establishing effective compliance and information technology (IT) controls is the focus. DevOps principles presented include mission first, customer focus, shift-left, continuous everything, and systems thinking. How stakeholders, including developers and operations staff, can collaborate and communicate effectively is described. The process outcomes and activities herein are aligned with the process model specified in ISO/IEC/IEEE 12207:2017 and ISO/IEC/IEEE 15288:2015.

REFERENCES

[1] IEEE Std, *ISO/IEC/IEEE 14764:2022, Software Engineering — Software Life Cycle Processes — Maintenance*, third ed.

[2*] P. Grubb and A.A. Takang, *Software Maintenance: Concepts and Practice*, 2nd ed. River Edge, NJ: World Scientific Publishing, 2003.

[3] A. April and A. Abran, *Software Maintenance Management: Evaluation and Continuous Improvement*. Wiley-IEEE Computer Society Press, 2008.

[4] C. Seybold and R. Keller, *Aligning Software Maintenance to the Offshore Reality*, 12th European Conference on Software Maintenance and Reengineering. April 1-4, 2008, Athens, Greece, DOI:10.1109/CSMR.2008.4493298.

[5] IEEE Standard for DevOps: *Building Reliable and Secure Systems Including Application Build, Package and Deployment*, IEEE Std 2675-2021, Apr. 2021.

[6] W. Titus, T. Manshreck, and H. Wright. *Software engineering at Google: Lessons learned from programming over time*. O'Reilly Media, 2020.

[7] A. Abran and H. Nguyenkim, Measurement of the maintenance process from a demand-based perspective, *Journal of Software Maintenance: Research and Practice*, Vol. 5 Issue 2: 63-90, 1993.

[8] M.M. Lehman, "Laws of software evolution revisited." *European workshop on software process technology*. Berlin, Heidelberg: Springer Berlin Heidelberg, 1996.

[9] J. Humble and D. Fairley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation,* Addison-Wesley, 2010.

[10] ISO/IEC/IEEE 12207:2017 *Systems and software engineering — Software life cycle processes*, 2017.